

Edu Platform — React + Django Roles & Architecture

1. Project Overview

Edu Platform is an educational platform developed for **Daltex School**.

The platform prepares students for entering the technology/job market through structured courses, instructors, lessons, assignments, quizzes, assessments, and performance tracking.

Technology Stack

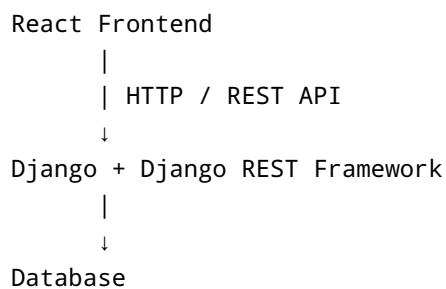
Frontend

- React
- React Router
- Tailwind CSS
- JavaScript

Backend

- Django
- Django REST Framework (DRF)
- Django authentication/authorization
- REST APIs

The frontend communicates with Django through REST API endpoints.



2. Main User Roles

The platform has exactly three primary roles:

ADMIN
INSTRUCTOR
STUDENT

The role should come from the authenticated Django user/account.

The React frontend must **not be the source of truth for permissions**.

Django must enforce all authorization rules on the backend.

React should only use the user's role to control the UI and navigation.

3. Admin

The Admin has the highest level of access.

Responsibilities

Admin manages the entire educational platform.

Admin can:

- Manage users.
- Manage students.
- Manage instructors.
- Create, edit, and delete courses.
- Assign instructors to courses.
- Manage enrollments.
- Monitor student performance.
- Monitor instructor activity.
- View platform statistics.
- Manage wallet/payment-related information.
- Manage platform settings.

Admin Dashboard

The Admin dashboard should provide information such as:

- Total students.
- Total instructors.
- Total courses.
- Active courses.
- Total enrollments.
- Student performance statistics.

- Platform activity.

Admin Routes

Suggested routes:

```
/admin  
/admin/users  
/admin/students  
/admin/instructors  
/admin/courses  
/admin/enrollments  
/admin/statistics  
/admin/settings
```

Only Admin users should be able to access these routes.

4. Instructor

The Instructor manages educational content and students enrolled in their courses.

Responsibilities

Instructor can:

- Manage their own courses.
- Create/manage lessons.
- Upload educational materials.
- Create assignments.
- Review submissions.
- Grade assignments.
- Create quizzes.
- Manage quiz questions.
- View student performance.
- Provide feedback.

Important Ownership Rule

An Instructor should only manage resources belonging to their courses.

Example:

```
Instructor A
|
+-- Course A → Can manage
|
+-- Course B → Cannot manage
```

Even if an Instructor manually sends a request to the API, Django must reject unauthorized operations.

Instructor Routes

Suggested routes:

```
/instructor
/instructor/courses
/instructor/courses/:id
/instructor/lessons
/instructor/assignments
/instructor/quizzes
/instructor/students
/instructor/performance
```

5. Student

The Student is the learner.

Responsibilities

Students can:

- Browse courses.
- View course details.
- Enroll in courses.
- Access enrolled courses.
- View lessons.
- Access learning materials.
- Complete lessons.
- Submit assignments.
- Take quizzes.
- View grades.
- Track progress.
- View performance.
- Manage their wallet.

- Receive notifications.

Student Ownership

A Student can only access their own private information.

For example:

```
Student A
|
+-- Own profile      → Allowed
+-- Own progress     → Allowed
+-- Own submissions  → Allowed
+-- Own grades       → Allowed
+-- Own wallet       → Allowed
|
+-- Student B grades → Forbidden
+-- Student B wallet → Forbidden
```

Django must enforce this at the API level.

Student Routes

Suggested routes:

```
/student
/courses
/courses/:id
/my-courses
/assignments
/quizzes
/progress
/wallet
/notifications
/profile
```

6. Permission Matrix

Feature	Admin	Instructor	Student
Login	✓	✓	✓

Feature	Admin	Instructor	Student
Register	✗/Depending on system	✗/Depending on system	✓
Manage Users	✓	✗	✗
Manage Students	✓	Own course students	✗
Manage Instructors	✓	✗	✗
Create Courses	✓	If allowed	✗
Edit Any Course	✓	✗	✗
Edit Own Course	✓	✓	✗
Manage Lessons	✓	Own courses	✗
Create Assignments	✓	Own courses	✗
Grade Assignments	✓	Own courses	✗
Create Quizzes	✓	Own courses	✗
Take Quizzes	✗	✗	✓
Submit Assignments	✗	✗	✓
View Student Performance	✓	Own students	Own performance
Enroll in Courses	✗	✗	✓
Manage Enrollments	✓	View own course enrollments	Own enrollment
Manage Wallet	✓	✗	Own wallet
Platform Statistics	✓	Own course statistics	Own statistics

7. Authentication

Django is responsible for authentication.

React should not implement its own independent authentication system.

The expected flow is:

```
User
↓
React Login Page
↓
Django Authentication API
↓
Django validates credentials
↓
Authentication established
↓
React receives authenticated user information
↓
React stores/maintains authentication state
```

The exact authentication mechanism should follow the existing Django backend implementation.

Do not invent a new authentication system if one already exists.

Before modifying authentication-related code, inspect the existing Django API and authentication implementation.

8. Authorization

Authorization is primarily handled by Django.

React should provide UI-level authorization:

```
Admin
  → Admin navigation

Instructor
  → Instructor navigation

Student
  → Student navigation
```

However, hiding a button is NOT security.

For example, React may hide:

Delete Course

from an Instructor who does not own the course.

But Django must also reject:

`DELETE /api/courses/123/`

if the Instructor does not have permission.

The correct architecture is:

```
React
↓
UI permission check
↓
Django API
↓
Authentication check
↓
Role permission check
↓
Object ownership check
↓
Database operation
```

9. Django Role Model

The backend should have a clear way of determining the user's role.

Possible implementation:

```
User
|
+-- role = ADMIN
+-- role = INSTRUCTOR
+-- role = STUDENT
```

The React application should consume the role returned by Django.

For example:

```
{
  "id": 15,
  "username": "mohamed",
  "email": "user@example.com",
  "role": "STUDENT"
}
```

Do not hardcode users or roles in the React application.

10. React Authentication Context

The React application should have a central authentication state, for example:

AuthContext

It should be responsible for information such as:

```
user
isAuthenticated
role
login()
logout()
```

Example conceptual structure:

```
const {
  user,
  isAuthenticated,
  role,
  login,
  logout
} = useAuth();
```

Components should use this information instead of duplicating authentication logic.

11. Protected Routes

React Router should protect frontend routes.

Examples:

```
ProtectedRoute  
AdminRoute  
InstructorRoute  
StudentRoute
```

Example:

```
<Route  
  path="/admin"  
  element={  
    <AdminRoute>  
      <AdminDashboard />  
    </AdminRoute>  
  }  
/>
```

Instructor:

```
<Route  
  path="/instructor"  
  element={  
    <InstructorRoute>  
      <InstructorDashboard />  
    </InstructorRoute>  
  }  
/>
```

Student:

```
<Route  
  path="/student"  
  element={  
    <StudentRoute>  
      <StudentDashboard />  
    </StudentRoute>  
  }  
/>
```

```
}  
/>
```

These routes improve the frontend experience but **must not replace Django permissions**.

12. API Integration

React should communicate with Django through a centralized API layer.

Avoid putting API requests randomly throughout components.

Prefer a structure such as:

```
src/  
├── api/  
│   ├── auth.js  
│   ├── courses.js  
│   ├── users.js  
│   ├── assignments.js  
│   ├── quizzes.js  
│   └── wallet.js  
├── components/  
├── context/  
├── pages/  
├── routes/  
└── utils/
```

For example:

```
api/auth.js  
api/courses.js  
api/assignments.js
```

This keeps API communication organized and easier to maintain.

13. API Error Handling

The React application should properly handle Django API responses.

Common cases:

```
200 → Success
201 → Created
400 → Bad Request
401 → Unauthenticated
403 → Forbidden
404 → Not Found
500 → Server Error
```

For example:

401

The user is not authenticated.

React should redirect the user to:

```
/login
```

403

The user is authenticated but does not have permission.

React should show an appropriate "Access Denied" page/message.

Do not treat `401` and `403` as the same thing.

14. Navigation by Role

The Navbar/sidebar should dynamically display the correct navigation.

Admin

```
Dashboard
Users
Students
Instructors
Courses
Enrollments
Statistics
```

Settings
Logout

Instructor

Dashboard
My Courses
Lessons
Assignments
Quizzes
Students
Performance
Notifications
Profile
Logout

Student

Dashboard
Browse Courses
My Courses
Assignments
Quizzes
Progress
Wallet
Notifications
Profile
Logout

Do not display navigation options that the current user cannot use.

15. Login/Register Pages

The `/login` and `/register` pages should have a different layout from authenticated pages.

The main Navbar should normally be hidden on:

`/login`
`/register`

Authenticated application pages can display the Navbar/sidebar.

16. Course Ownership

Course permissions are especially important.

Admin

Can manage all courses.

Instructor

Can manage only courses assigned to them.

Student

Can view courses according to enrollment/access rules but cannot modify course content.

Example:

```
Course
├── Instructor
├── Students
├── Lessons
├── Assignments
└── Quizzes
```

The backend should verify ownership/relationship before allowing modifications.

17. Development Rules for Codex

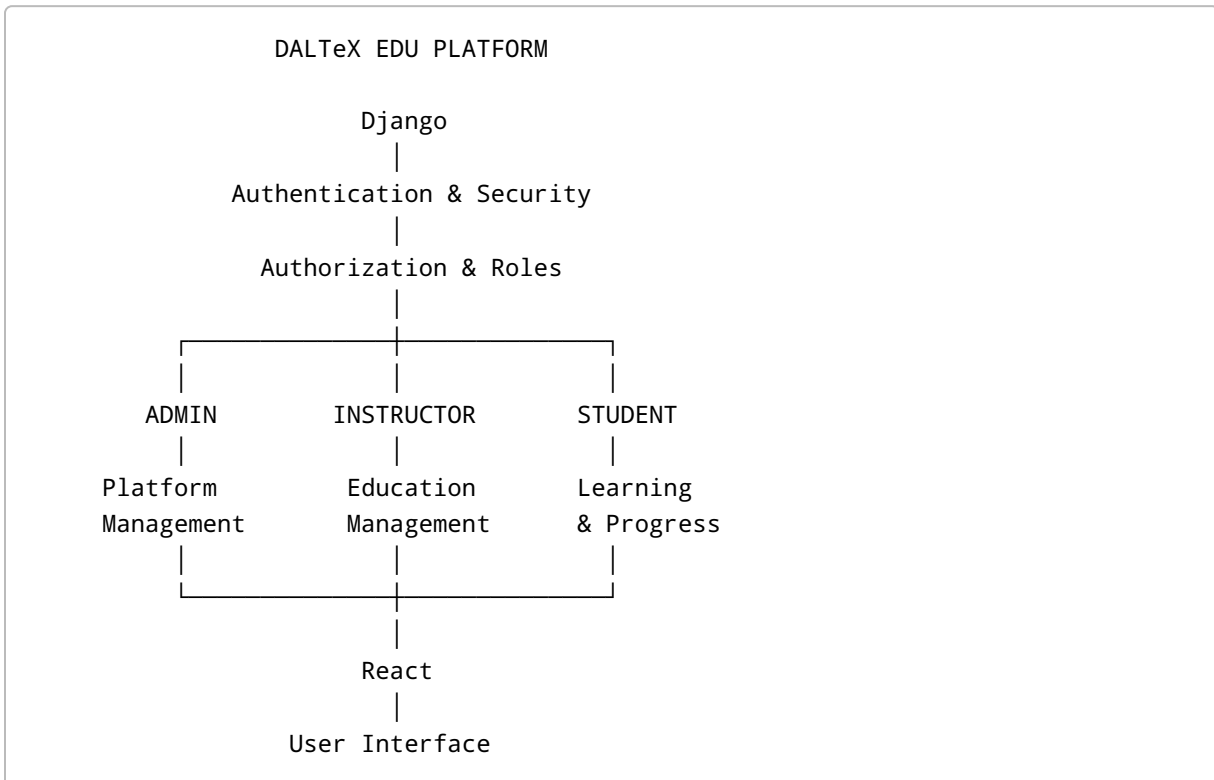
Before implementing any new feature:

1. Inspect the existing React code.
2. Inspect the relevant Django API endpoints.
3. Identify which role should have access.
4. Check existing authentication and authorization logic.
5. Reuse existing components and services where possible.
6. Do not create duplicate authentication logic.
7. Do not hardcode roles.
8. Do not hardcode API URLs if the project already uses environment variables.

9. Keep API calls separated from UI components when practical.
10. Keep role-based UI consistent throughout the application.
11. Never rely on frontend authorization for security.
12. Make sure Django enforces the same permission rules.

18. Core Principle

The platform follows this responsibility model:



Django = source of truth for data, authentication, authorization, and security.

React = user interface, routing, state management, and user experience.

Any new feature must respect this architecture.